DarkGuard AI: Technical Requirements Document (v3, Hackathon Scope)

This document describes how the MVP defined in the PRD is built. Section numbers refer to this document unless noted.

1. Technology stack

Layer	Technology	Reason
Extension	JavaScript (ES modules), Chrome Extension Manifest V3	Native browser APIs, works in Chrome and Edge
Rule detectors	Plain JavaScript modules, run in the content script	Fast, offline, easy to test with fixtures
Text classifier (inference)	Plain JavaScript logistic regression over TF-IDF features, weights in JSON	No runtime dependency, under 5 ms, auditable
Text classifier (training)	Python 3, scikit-learn, pandas	Standard, reproducible baseline
Popup UI	HTML, CSS, vanilla JS	Small surface, no build step
Landing page	React + Tailwind CSS (static build)	Quick to deploy on Vercel
Test site	Static HTML/JS with planted patterns	Ground truth for evaluation
Optional backend	Python + FastAPI	Only for AI-written explanation text (`/v1/explain`)
Storage	`chrome.storage.local`	Scan history and settings, no account
Tests	Jest (rules, risk, classifier parity), Playwright (end-to-end on test site), pytest (training pipeline)	Covers logic, browser behavior, and reproducibility

Not used: transformer models at runtime, generative models for page changes, PostgreSQL, OpenCV, React dashboard.

2. Extension permissions

```
{
  "manifest_version": 3,
  "name": "DarkGuard AI",
  "version": "0.3.0",
  "permissions": ["activeTab", "scripting", "storage"],
  "optional_host_permissions": ["https://*/*", "http://*/*"],
  "action": { "default_popup": "popup/popup.html" },
  "background": { "service_worker": "background/service-worker.js", "type": "module" },
  "web_accessible_resources": [
    { "resources": ["content/ethical-view.css"], "matches": ["<all_urls>"] }
  ],
  "content_security_policy": { "extension_pages": "script-src 'self'; object-src 'self'" }
}
```

- `activeTab` grants access to the current tab only after the user clicks the extension.
- `optional_host_permissions` is requested per origin when the user scans a site.
- No `cookies`, `history`, `tabs` (all URLs), `webRequest`, or remote code.

3. Project structure

```
darkguard-ai/
├── extension/
│   ├── manifest.json
│   ├── popup/                  popup.html, popup.css, popup.js
│   ├── background/             service-worker.js
│   ├── content/
│   │   ├── scanner.js          extracts candidate elements
```

```
│   │   ├── features.js         tokenizer and TF-IDF features (shared with training)
│   │   ├── classifier.js       loads model, runs inference, checksum check
│   │   ├── detectors/          rule modules: dp1-urgency.js ... dp4-extras.js
│   │   ├── fusion.js           combines rule evidence with classifier output
│   │   ├── risk.js             score calculation
│   │   ├── highlighter.js      outlines and scroll-to
│   │   ├── ethical-view.js     applies and restores templates
│   │   ├── templates.json      Ethical View catalogue (appendix A)
│   │   ├── detectors/models/   shaming-v1.json, model-card-v1.md
│   │   └── ethical-view.css
│   ├── services/api-client.js  optional backend calls
│   └── tests/                  fixtures, unit tests, parity tests
├── ml/
│   ├── data/                   labeled strings (CSV), split manifests
│   ├── labeling.md             labeling guidelines
│   ├── train.py                trains baseline, writes JSON artifact
│   ├── export.py              writes weights, vocabulary, checksum
│   ├── evaluate.py             metrics, ablation, release gate
│   ├── golden/                 expected features for parity tests
│   └── notebooks/              exploration (optional)
├── landing/                    React + Tailwind site
├── test-site/                  mock shop with planted and clean pages
├── backend/                    optional FastAPI service
├── eval/
│   ├── labels.json             page-level ground truth
│   ├── run.js                  rules-only vs rules-plus-ML report
│   └── reports/
└── docs/                       PRD.md, TRD.md, ARCHITECTURE.md
```

---

4. Content script pipeline

Each scan runs the same pipeline in the content script:

```
scanner.js  ->  candidates
   |
   +-> rule detectors (DP-1, DP-2, DP-4, plus DP-3 phrase check)  ->  rule evidence
   |
   +-> classifier.js (text candidates only)                        ->  category probabilities
   |
   v
fusion.js   ->  detections (with source: rule | ml | rule+ml)
   |
   v
risk.js     ->  score and level
   |
   v
ethical-view.js (only if enabled) -> approved templates only
```

4.1 Candidate extraction (`scanner.js`)

Candidate shape

```
{
  "ref": "dg-17",
  "tag": "button",
  "role": "button",
  "text": "No, I don't want to save money",
  "attributes": { "type": "submit", "disabled": false },
  "rect": { "x": 412, "y": 830, "w": 180, "h": 44 },
  "selector": "div.cart-footer > button:nth-of-type(2)",
  "inForm": true,
  "nearPrimaryAction": true,
  "visible": true
}
```

Rules

- Visible elements only (`display`, `visibility`, `opacity`, size checked).
- Text trimmed to 200 characters and whitespace-normalized.
- Input values are never read. For checkboxes and radios, only `checked` is recorded.
- Password, email, phone, card, and address inputs are skipped entirely.
- `data-dg-ref` is written to each candidate for highlighting.

4.2 Detector interface

```
export const detector = {
  id: "DP-1",
  category: "fake_urgency",
  run(ctx) {
    // ctx: { candidates, priceNodes, document, observeMs }
    // returns RuleEvidence[]
  }
};
```

---

5. Rule detectors

Rules produce evidence. Each rule finding has `source: "rule"`.

DP-1 Fake urgency

- Find elements whose text matches a time pattern (`\d{1,2}:\d{2}(:\d{2})?`, or "days/hours/minutes") near urgency words.
- Record the value, wait `observeMs` (default 10,000 ms), and read again.
- Unchanged value: rule evidence `frozen_countdown`, confidence 0.9.
- Urgency wording with no timer or stock indicator: rule evidence `urgency_text_only`, confidence 0.5. This evidence is passed to the classifier for a second opinion (section 6.3).

DP-2 Misleading discount

- Parse prices with a locale-aware regex (currency symbols, , and . separators).
- Find a reference price marked with `<s>`, `<del>`, `line-through`, or a "was", "MRP", or "list price" label.
- Find a stated discount percentage.
- Calculated discount = `(reference − current) / reference × 100`.
- Difference greater than 5 points: rule evidence `arithmetic_mismatch`, confidence 0.9.
- Reference price present but unlabeled: rule evidence `unlabeled_reference`, confidence 0.5.

DP-3 Confirm shaming (phrase check)

- Decline-style buttons or links in the same container as a primary action.
- Curated phrase list (`phrases/shaming.json`, about 40 English phrases) matched as lowercase substrings.
- Match: rule evidence `shaming_phrase`, confidence 0.7.

DP-4 Pre-selected extras

- Checkboxes that are `checked` by default.
- Label or nearby text matches add-on keywords (insurance, protection, warranty, gift wrap, donation, subscription).
- Checkbox sits in the same form or product-action container as an add-to-cart or buy control.
- Rule evidence `preselected_addon`, confidence 0.85.

---

6. Text classifier

6.1 Purpose and scope

The classifier estimates, for a short UI string, the probability that it belongs to each text-based category. It supports DP-3 most directly and adds signal for DP-1, DP-2, and DP-4 where wording varies.

Scope: English strings under 200 characters, from candidates only (buttons, links, labels, banner text, nearby headings). It never reads full page text.

6.2 Labels

Multi-label, one probability per class:

Class	Meaning
`urgency`	Pressure or scarcity wording
`shaming`	Guilt or self-directed pressure in a decline option
`addon_extra`	Label of an optional add-on or extra
`discount_wording`	Discount or savings claim
`neutral`	None of the above (used for training, not scored)

6.3 Model

Architecture

- Features: word unigrams and bigrams, plus character 3–5-grams, TF-IDF weighted, sublinear TF, maximum 8,000 features.
- Classifier: one-vs-rest logistic regression per class, `class_weight="balanced"`, L2 regularization with C tuned on the validation split.
- Output: `p(class | text)` for each class, in [0, 1].

Inference (JavaScript, `classifier.js`)

```
score_c = sigmoid( b_c + Σ_j w_c,j × tfidf_j(text) )
```

- Tokenization and n-gram generation are implemented in `features.js`. Training uses the same rules, written in Python, and parity tests check that both produce identical feature vectors for the golden set (`ml/golden/`).
- Sparse vector: stored as index-value pairs in the JSON, so inference is a dot product over non-zero features.
- Latency budget: under 5 ms for 200 strings. Measured in the Jest suite.

Artifact: `shaming-v1.json`

```
{
  "model_id": "darkguard-text-v1",
  "trained_at": "2026-11-15T10:00:00Z",
  "training_data_sha256": "…",
  "features": { "type": "tfidf", "ngram_range": [[1,2],[3,5]], "analyzer": ["word","char_wb"],
    "lowercase": true, "sublinear_tf": true, "vocab_size": 7842 },
  "vocabulary": { "no thanks": 0, "save money": 1, "…": "…" },
  "idf": [1.2, 3.4, "…"],
  "classes": ["urgency", "shaming", "addon_extra", "discount_wording"],
  "weights": { "urgency": { "indices": [0, 5], "values": [1.1, -0.3] }, "…": {} },
  "intercepts": { "urgency": -2.1, "…": 0.0 },
  "thresholds": { "urgency": 0.6, "shaming": 0.7, "addon_extra": 0.65, "discount_wording": 0.6
    },
  "checksum_sha256": "…"
}
```

- `checksum_sha256` covers the file contents excluding the checksum field. `classifier.js` verifies it before use.
- Thresholds are set on the validation split to meet the precision target for each class.

6.4 Fusion with rules (`fusion.js`)

Case	Result	Confidence	Ethical View
Rule evidence present and classifier agrees above threshold	Detection, `source: rule+ml`	Rule confidence, raised by up to 0.05 if classifier is above 0.9 (cap 0.95)	All approved templates for the category
Rule evidence present, classifier disagrees or is unavailable	Detection, `source: rule`	Rule confidence	All approved templates for the category
No rule evidence, classifier above threshold	Signal, `source: ml`	Classifier probability capped at 0.5	Caption templates only (`ET-SIGNAL-01`)
No rule evidence, classifier below threshold	Nothing reported	—	—

Case	Result	Confidence	Ethical View
Classifier fails to load or checksum fails	Rules only	Rule confidence	As above; popup shows "text model unavailable"

Rule evidence always takes precedence. A classifier cannot create a strong detection by itself.

6.5 Release gate

The model is enabled by default only if all of these hold on the held-out data (`ml/evaluate.py`):

- Precision ≥ 0.85 on the held-out clean set for each class that is enabled.
- Recall ≥ 0.70 on the planted set for the classes it supports.
- False-positive rate on clean pages with rules plus ML is not higher than rules alone.
- The guarantee tests (section 9.4) pass.

If a gate fails, the model ships disabled and the report says why.

6.6 Model card

Each version includes `model-card-v1.md` with: intended use, out-of-scope uses, training data source and license, split method, per-class metrics on held-out data, per-site results, known failure cases (for example, non-English text, sarcasm, and unusual layouts), and the release gate result.

---

7. Risk scoring

Each category contributes at most once. The score uses the category's rule confidence, or the fused confidence where the classifier contributed.

For each category `c` with weight `W_c`:

- `S_c` = highest severity among that category's detections, divided by 3.
- `C_c` = highest confidence among those detections.
- Contribution = `W_c × S_c × C_c`.

```
Risk = 100 × Σ_c ( W_c × S_c × C_c ) / Σ_c W_c
```

The sum covers all four categories. Weights sum to 1 and are stored in `risk.js`:

Category	Weight
DP-1 Fake urgency	0.30
DP-2 Misleading discount	0.30
DP-3 Confirm shaming	0.15
DP-4 Pre-selected extras	0.25

Classifier-only signals (capped at 0.5) contribute at most half their category's weight. They raise the score but cannot move a page into a high band by themselves.

Example: a single frozen timer at confidence 0.9 and high severity gives `100 × 0.30 × 1 × 0.9 = 27` (Moderate).

Score	Level
0–24	Low
25–49	Moderate
50–74	High
75–100	Very high

Levels are presented as signals about the page, not as legal conclusions.

---

8. Ethical View engine

8.1 Behavior

- Templates come only from `templates.json` (appendix A). Neither page content nor the classifier can add templates.
- Category-to-template mapping is a lookup table in `templates.json`. There is no runtime model choosing templates.
- Actions: `outline`, `caption`, `banner`, `comparison`. Each is implemented in `ethical-view.js`.
- Placeholders are filled only from validated evidence fields. A missing value causes that annotation to be skipped.
- Inserted nodes carry `data-dg-owner="ethical-view"` and `data-dg-id`.
- Original styles are captured before any change and restored exactly.

8.2 Apply and restore

```
apply(detections):
  snapshot = captureStyles(candidates)
  for each detection:
      template = lookup(detection.category, detection.source)
      if template: render(template, detection)
  insert banner with "Show original" toggle
  state = ON

restore():
  remove nodes where data-dg-owner = "ethical-view"
  reapply snapshot styles
  remove banner
  state = OFF
```

If `apply` throws partway, it calls `restore` automatically.

8.3 Dynamic pages

A `MutationObserver` watches candidate containers. Changes trigger a scan after a 500 ms debounce. Ethical View is reapplied only to new detections. The observer is disconnected when Ethical View is off.

8.4 Guarantees enforced in code

- The engine rejects any template whose action would change `value`, `checked`, `disabled`, `href`, `action`, or price text. A unit test checks every template in `templates.json`.
- The engine never calls `click()`, `submit()`, or `dispatchEvent` on page controls.
- Annotations use `textContent`. No `innerHTML` with page-derived strings.

---

9. Data model and storage

9.1 Scan record

```
{
  "scans": [
    {
      "id": "scan-2026-11-15-1432",
      "domain": "testshop.example",
      "url_path": "/products/desk-lamp",
      "timestamp": "2026-11-15T14:32:00Z",
      "risk_score": 52,
      "level": "High",
      "model_version": "darkguard-text-v1",
      "ml_enabled": true,
      "detections": [
        { "category": "DP-1", "severity": 3, "confidence": 0.9, "source": "rule" },
        { "category": "DP-3", "severity": 2, "confidence": 0.7, "source": "rule+ml" }
      ]
```

```
    }
  ]
}
```

- Domain and path only. Query strings are dropped.
- Last 20 scans kept. Older scans are deleted automatically.
- Scan records never include page text beyond the short evidence snippet (≤ 200 characters).

9.2 Training data files

`ml/data/strings.csv` columns: `id`, `text`, `labels` (pipe-separated), `site_id`, `split` (train/val/test), `source` (planted, clean, hand-written, licensed-dataset), `license`, `labeler_1`, `labeler_2`, `agreed`.

Rules:

- Split by `site_id`, never by row.
- Two labelers per string. Disagreements are resolved and recorded in `agreed`.
- No personal data. No form values. No full HTML.

---

10. Optional backend API

Method	Endpoint	Purpose
GET	`/v1/health`	Liveness check
POST	`/v1/explain`	Rewrites template explanation text

- HTTPS in production. CORS allows only the extension origin.
- Rate limit: 30 requests per minute per IP.
- Pydantic schemas. Unknown fields are rejected.
- Request payload contains detection summaries only (category, rule, evidence snippet). No page HTML.
- The LLM output is plain text. It is never executed or inserted as HTML.
- The backend does not run the classifier. Detection never depends on the backend.

---

11. Training pipeline (offline)

```
ml/data/strings.csv
   -> train.py      (fit TF-IDF and logistic regression on train split)
   -> evaluate.py   (validation thresholds, test metrics, ablation, release gate)
   -> export.py     (write shaming-v1.json, compute checksum, write model card stub)
   -> golden test   (feature parity check between Python and JavaScript)
```

Reproducibility:

- Fixed random seeds.
- Package versions pinned in `ml/requirements.txt`.
- `train.py` records the SHA-256 of the input CSV in the artifact.

Ablation report (`eval/reports/ablation.md`): for each category, rules-only versus rules plus ML on the planted and clean sets, with precision, recall, and false-positive rate.

---

12. Testing and evaluation

12.1 Unit tests (Jest)

- Each rule detector tested against HTML fixtures, with positive and negative examples.
- Price parsing for `₹1,299`, `$19.99`, `1.299,00 €`.
- Risk score boundaries: all zero, all maximum, single category.
- Ethical View apply then restore leaves the DOM identical to the snapshot.

- Guarantee tests (section 8.4) for every template.
- Fusion table (section 6.4): each row has a test.

12.2 Classifier tests

- **Parity:** JavaScript and Python produce identical features for every string in `ml/golden/`.
- **Latency:** 200 strings in under 5 ms.
- **Checksum:** tampered artifact is rejected and rules-only mode is used.
- **Fallback:** missing artifact results in rules-only scan and the "text model unavailable" message.

12.3 End-to-end tests (Playwright)

1. Planted page: scan, expect the planted detections.
2. Clean page: scan, expect no detections.
3. Ethical View on, then off: prices and controls unchanged, DOM identical to the original.
4. Classifier disabled in settings: scan still completes.

12.4 Labeled evaluation set

- `eval/labels.json` lists each page, expected detections, and whether it is planted or clean.
- At least 30 pages. Each planted category appears at least 4 times.
- `eval/run.js` writes precision, recall, and false-positive rates per category for both rules-only and rules-plus-ML.

---

13. Security and privacy

- No page content leaves the device by default.
- No reading or storing of passwords, payment fields, addresses, or other form values.
- Domain and path only in history.
- Page text is untrusted. It is never executed and never used to build templates.
- No `eval`, no `innerHTML` with page strings, no remote code.
- Model artifact integrity via SHA-256 checksum on every load.
- The backend stores no scan data and logs no page content.
- Training data is limited to short UI strings and labels. Third-party data is used only under its license, with the source recorded in the model card.

Appendix A: Ethical View template catalogue

File: `extension/content/templates.json`

Placeholders in `{braces}` are filled only from validated evidence. Wording is neutral and describes observations.

```
{
  "version": "1.1",
  "lookup": {
    "fake_urgency": ["ET-URG-01", "ET-URG-02"],
    "misleading_discount": ["ET-DISC-01", "ET-DISC-02"],
    "confirm_shaming": ["ET-CONF-01"],
    "preselected_extra": ["ET-EXTRA-01"],
    "classifier_signal": ["ET-SIGNAL-01"]
  },
  "templates": [
    {
      "id": "ET-URG-01",
      "action": "caption",
      "placement": "after",
      "text": "Timer check: this countdown did not change during a {observed_seconds}-second
    check.",
      "style": "warning"
    },
    {
```

```
      "id": "ET-URG-02",
      "action": "caption",
      "placement": "after",
      "text": "Urgency message: no visible stock or deadline indicator on this page supports
    this claim.",
      "style": "info"
    },
    {
      "id": "ET-DISC-01",
      "action": "comparison",
      "placement": "after",
      "text": "Price check: current price {current_price}, reference price {reference_price}.
    The page states {stated_percent}% off; the calculated discount is {calculated_percent}%.",
      "style": "warning"
    },
    {
      "id": "ET-DISC-02",
      "action": "caption",
      "placement": "after",
      "text": "Reference price: the higher price is shown without a clear label. Check what it
    refers to before comparing.",
      "style": "info"
    },
    {
      "id": "ET-CONF-01",
      "action": "caption",
      "placement": "after",
      "text": "Wording note: this option uses persuasive language. Choosing it still performs
    the button's original action.",
      "style": "info"
    },
    {
      "id": "ET-EXTRA-01",
      "action": "caption",
      "placement": "before",
      "text": "Pre-selected extra: this option was checked by default. Review it before
    continuing.",
      "style": "warning"
    },
    {
      "id": "ET-SIGNAL-01",
      "action": "caption",
      "placement": "after",
      "text": "Possible pattern (text model, low certainty): this wording may be persuasive.
    Review in context.",
      "style": "info"
    },
    {
      "id": "ET-BANNER-01",
      "action": "banner",
      "placement": "top",
      "text": "DarkGuard: {count} possible issue(s) found on this page. Risk score {risk_score}
    ({level}).",
      "controls": ["Show original", "Details"],
      "style": "neutral"
    }
  ]
}
```

Action meanings:

- `outline`: highlights the element with an outline. Content is unchanged.
- `caption`: inserts a small note beside the element.
- `comparison`: shows a small panel with the two prices. Neither price on the page is changed.
- `banner`: summary at the top of the page, with the "Show original" toggle.

No template changes, hides, or disables a control.

Appendix B: Element locator

Highlighting and scroll-to use `data-dg-ref`, written during extraction. The CSS selector is stored only for reporting. If the element is no longer in the DOM, the highlight is skipped and the report marks the

evidence "no longer on page."

Appendix C: Model card template

```
# Model card: darkguard-text-v1

- Intended use: flag short UI strings that may belong to dark-pattern categories, as a signal
    alongside rule evidence.
- Out of scope: legal findings, non-English text, full-page analysis, any decision that changes
    page content directly.
- Training data: <source, size, license, labelers, agreement rate>
- Split: by site. Sites in test set do not appear in training.
- Metrics (held-out): per-class precision, recall, F1, ROC AUC.
- Per-site results: <table>
- Known failure cases: <list>
- Release gate: <pass/fail, with the numbers>
- Version history: <dates and changes>
```